



第十一届中国数据库技术大会

DATABASE TECHNOLOGY CONFERENCE CHINA 2020

架构革新 高效可控



北京国际会议中心 | 2020/12/21-12/23

有赞每秒万级订单的DB优化实践

杨一 @有赞 DBA



- 1.介绍
- 2.核心问题
- 3.如何解决
- 4.总结

自我介绍

2015至今
有赞 DBA





香港上市代码
08083.HK

首页

商家服务 ▾

消费者 ▾

渠道合作 ▾

有赞云 ▾

更多服务 ▾

搜一搜



商家后台

免费试用



致电领取开店礼包
0571-8685 7988

产品



有赞微商城

网上开店工具，帮你提升销量



有赞零售

打通门店网店，业绩翻倍增长



有赞连锁

多门店统一经营解决方案



有赞美业

轻松拉新客，快乐管店务



有赞教育

教育机构经营增长解决方案

更多产品...

服务一切交易

解决方案

微信小程序

一键接入，抢占市场红利

支付宝小程序

快速上线，在支付宝生态开店卖货

QQ小程序

一键上线，在QQ群/QQ空间做生意

分佣推广（有赞客）

CPS 推广，网红主播帮你打造爆款

分销

搭建分销网络，万人帮你卖货

营销应用

百种营销工具，玩转社交电商

餐饮

外卖、堂食，统一高效管理

票务

在线售票，快速核销，提升营收能力

媒体

快速拥有网店，微信粉丝变现

酒店

满足订房、餐饮、娱乐等多场景

会议

一站式全流程会务解决方案



有赞护航

系统稳定高于一切

直播

搭建直播店铺，实现流量变现

App 开店

让每个 App 都拥有强大的商城能力

广告投放

连接优质流量，获取目标客户

定制服务

个性化需求，独立定制

更多解决方案...

帮助更多垂直行业的商家成功

543 万

商家信赖之选

150 个

覆盖行业和类目

4 亿

在售商品数

645 亿

商家2019年交易额

6 万/秒

订单处理速度

DTCC
2020



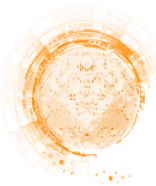
北京国际会议中心

2020/12/21-12/23

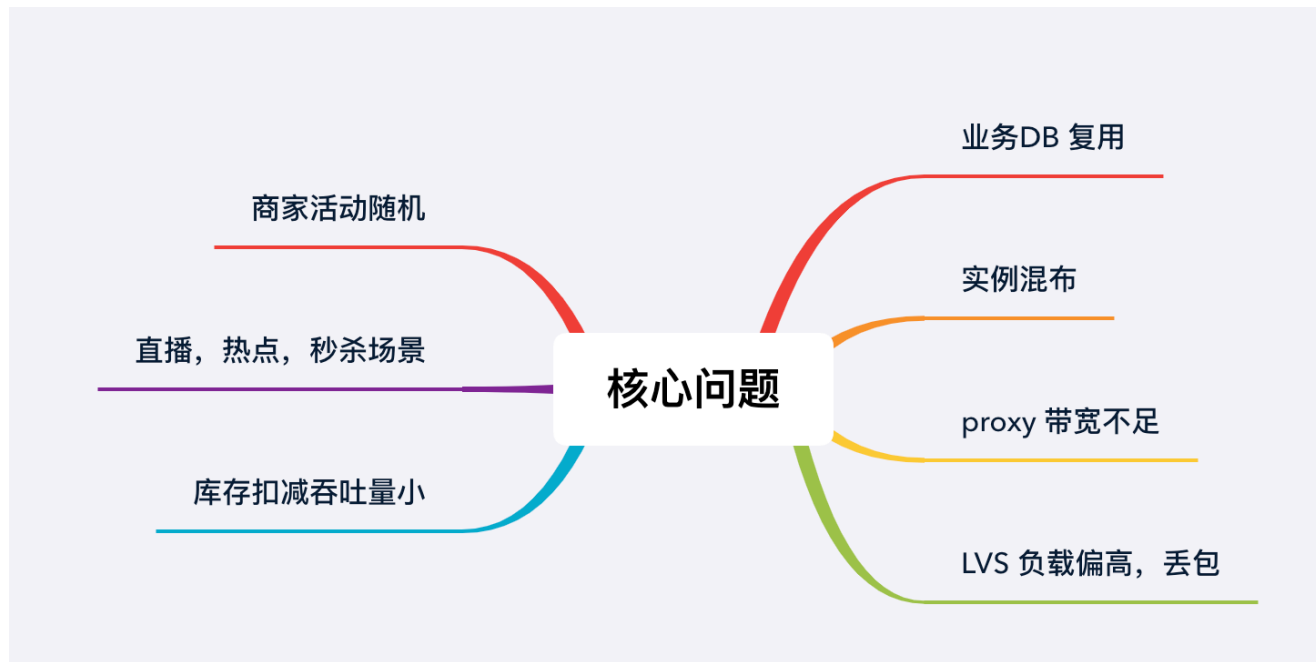


ChinaUnix.net

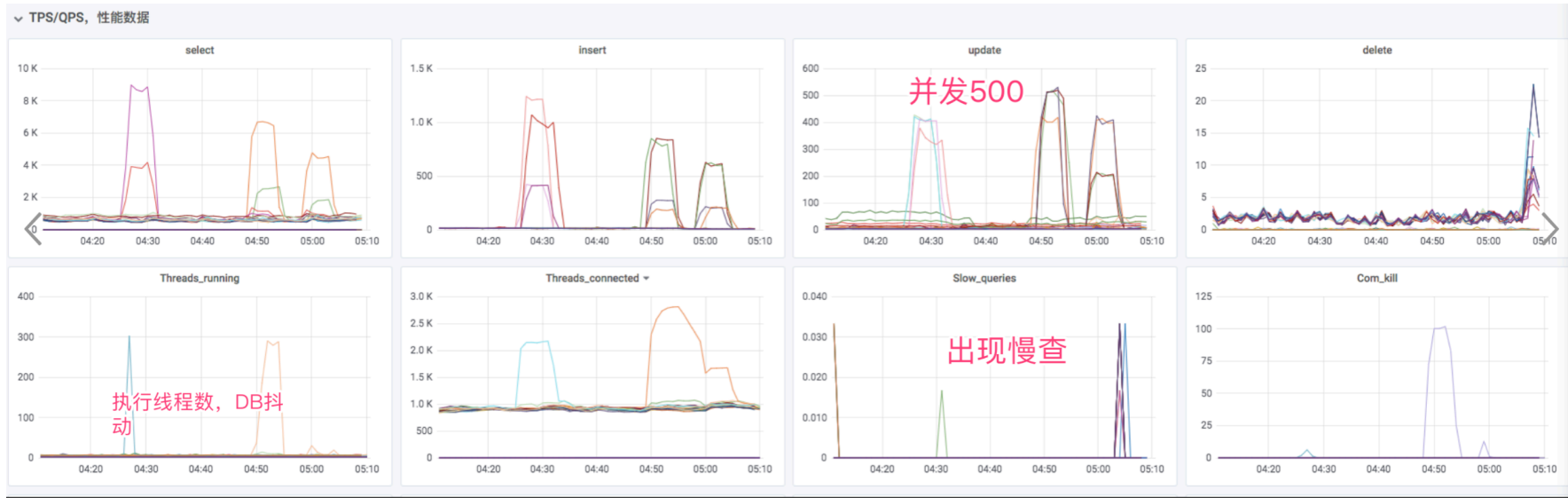
ITPUB

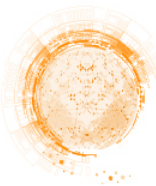


核心问题

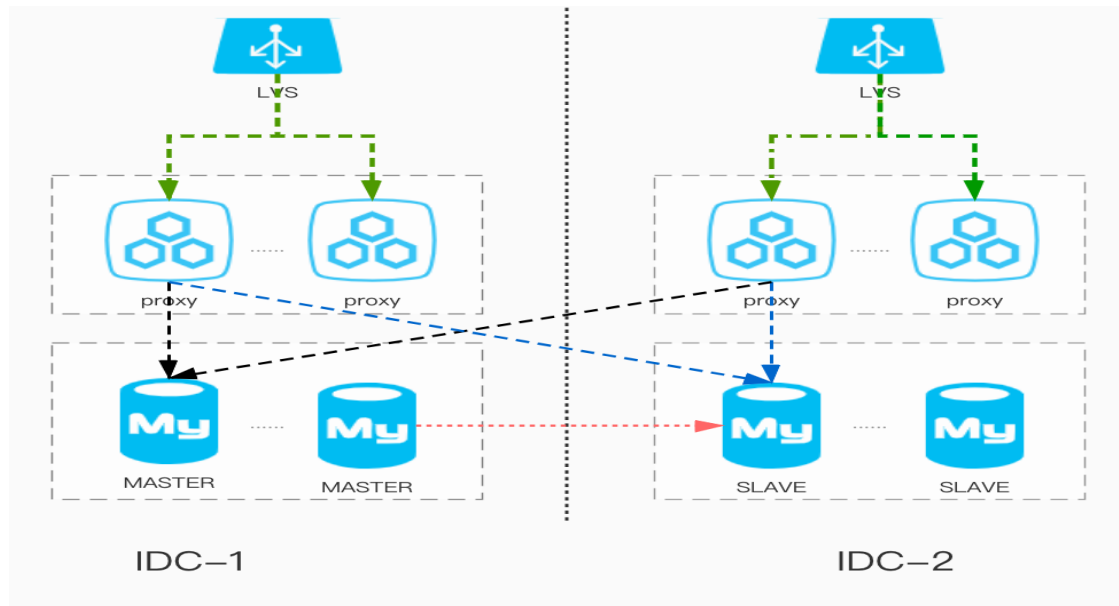


优化之前

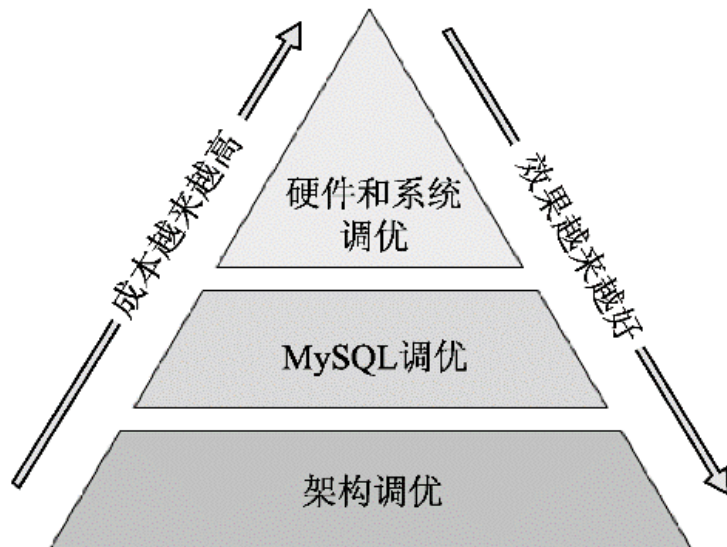




访问链路



如何解决





如何解决-核心思想

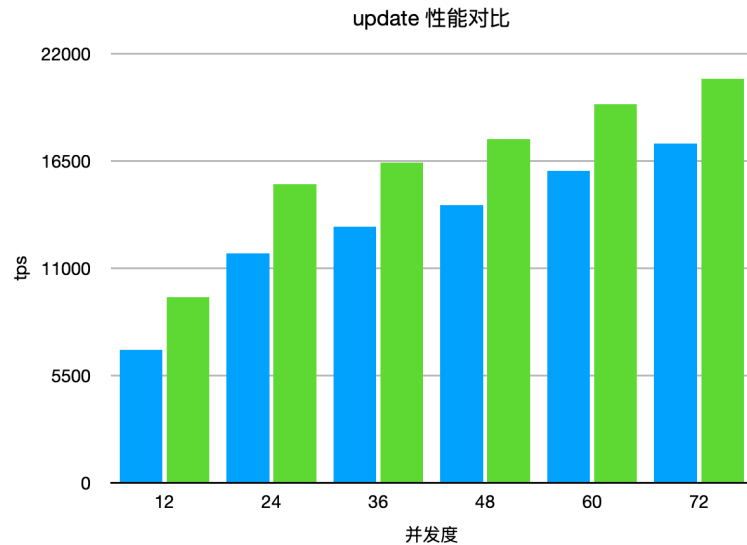
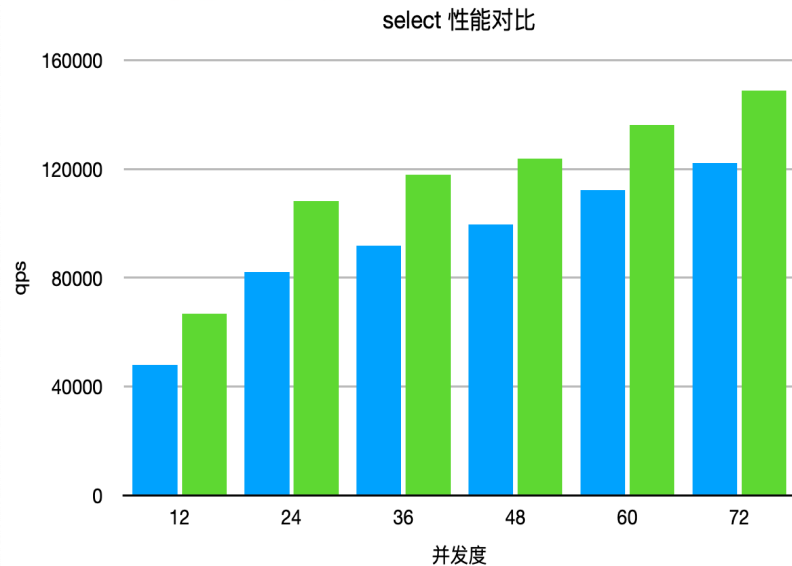
- 减少锁竞争
- 控制并发
- 降低RT ,提高吞吐量

如何解决-系统层

- 拓展 CPU 核数 从56core 增加到80core
- 使用处理能力更强的CPU 2.4GHz — 2.6GHz
- 文件系统 ext4 替换为 xfs
- 升级操作系统 和内核版本
- 优化网络链路



架构革新 ◎ 高效可控
第十一届中国数据库技术大会
DATABASE TECHNOLOGY CONFERENCE CHINA 2020



如何解决-DB 层

- 垂直拆分: 独立交易链路DB集群
- 水平拓展: 库存 支付业务拆分
- 优化SQL: 去掉非必须的大字段,减少带宽,降低RT
- 参数优化: 关闭死锁检测, 动态修改落盘策略

核心优化参数

```
sync_binlog=0           -- 交给系统来刷binlog
innodb_flush_log_at_trx_commit=0 -- 每秒刷新redo buffer到磁盘
innodb_deadlock_detect=OFF --关闭死锁检测
innodb_lock_wait_timeout=2
```

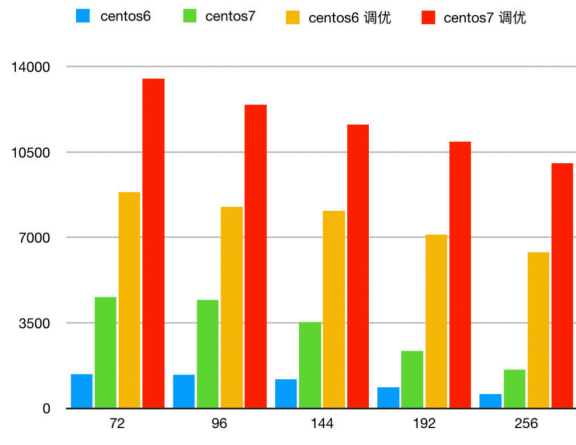
压测场景:

```
CREATE TABLE `seckill` (
  `id` int(11) DEFAULT NULL,
  `num` bigint(20) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

insert into seckill(id,num) values(1,2000000000000000);

mysqlslap -uroot -h127.0.0.1 -P 3316 --concurrency=72 --create-schema='test'
```

并发数	centos6	centos7	centos6 调优	centos7 调优
72	1388	4568	8857	13503
96	1374	4438	8256	12426
144	1188	3534	8081	11630
192	857	2353	7116	10928
256	586	1579	6384	10051



如何解决-Proxy 层

- 扩容 Proxy 实例
 - 开发限流功能
 - 优化连接池
- 增加吞吐量
基于表，行限流
减少获取连接的耗时

Proxy 层--限流

* 产品类型: rds

* 资源类型①: 热key限流

统计模式: 单机

限流策略: 静态限流

* 规则名称:

* 限流特征:

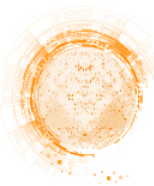
* 限流动作: 排队

* 最大等待时间(ms): 800

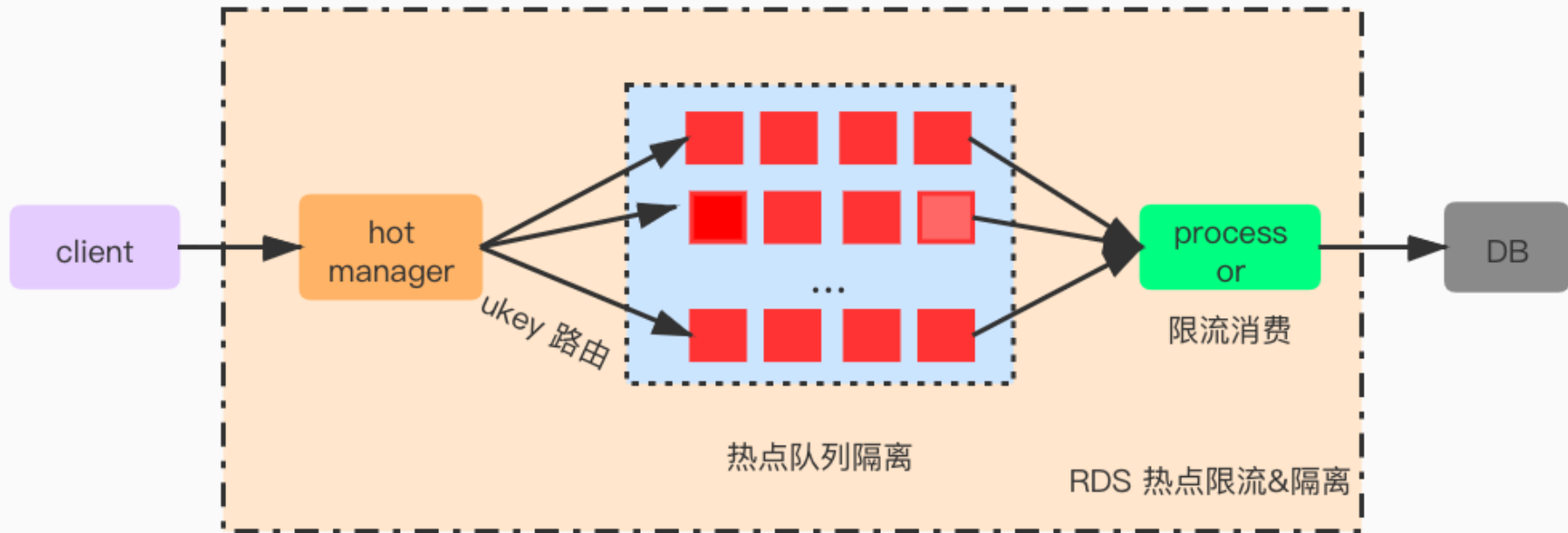
* 队列长度: 100

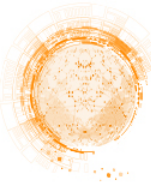
* 阈值: 8

```
/*!ctx:hot_update, body:{"ukey":"xxx", "table":"xxx"}*/  
update pressure_test set num = num;
```



Proxy 层--限流

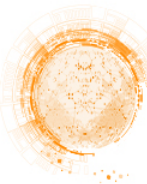




Proxy 层—优化网络

3271	12:58:25.566531	10.96		10.25	MySQL	8754	ResponseResponse
3276	12:58:25.568040	10.25		10.96	TCP	66	36881 → 3335 [ACK] Seq=3929 Ack=95412 Win=757 L
3278	12:58:25.568147	10.25		10.96	TCP	66	36881 → 3335 [ACK] Seq=3929 Ack=102652 Win=757 L
3279	12:58:25.568150	10.96		10.25	MySQL	8754	ResponseResponse
3287	12:58:25.569847	10.25		10.96	TCP	66	36881 → 3335 [ACK] Seq=3929 Ack=111340 Win=757 L
3288	12:58:25.569857	10.96		10.25	MySQL	8754	ResponseResponse
3297	12:58:25.571410	10.25		10.96	TCP	66	36881 → 3335 [ACK] Seq=3929 Ack=118580 Win=757 L
3298	12:58:25.571415	10.96		10.25	MySQL	8754	ResponseResponse
3310	12:58:25.572997	10.25		10.96	TCP	66	36881 → 3335 [ACK] Seq=3929 Ack=128716 Win=711 L
3311	12:58:25.573002	10.96		10.25	MySQL	8754	ResponseResponse
3320	12:58:25.574576	10.25		10.96	TCP	66	36881 → 3335 [ACK] Seq=3929 Ack=137404 Win=757 L
3321	12:58:25.574582	10.96		10.25	MySQL	8754	ResponseResponse
3518	12:58:25.616699	10.25		10.96	TCP	66	36881 → 3335 [ACK] Seq=3929 Ack=146092 Win=757 L
3519	12:58:25.616709	10.96		10.25	MySQL	4410	ResponseResponse
3520	12:58:25.616715	10.96		10.25	MySQL	4410	Response [TCP segment of a reassembled RBU]

delay ack

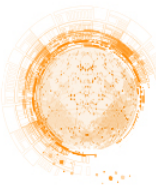


Proxy 层—优化网络

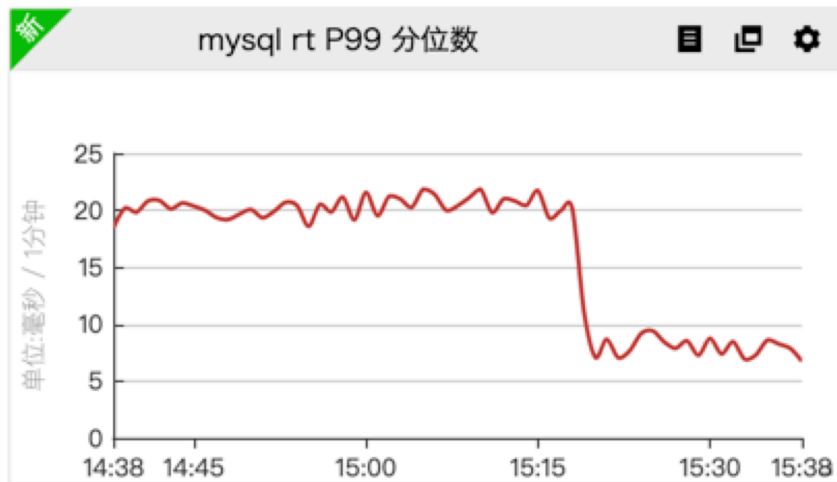
- TCP延迟确认（TCP delayed acknowledgment）

传输控制协议的一些实现所使用的技术，用于改善网络性能。该技术可以在本质上将若干ACK报文组合在一起成为单个报文，从而减少协议开销。但是，在某些情况下，该技术可能会降低应用程序性能。

- 控制参数：
tcp_delayed_ack
tcp_ato_min



Proxy 层—优化网络

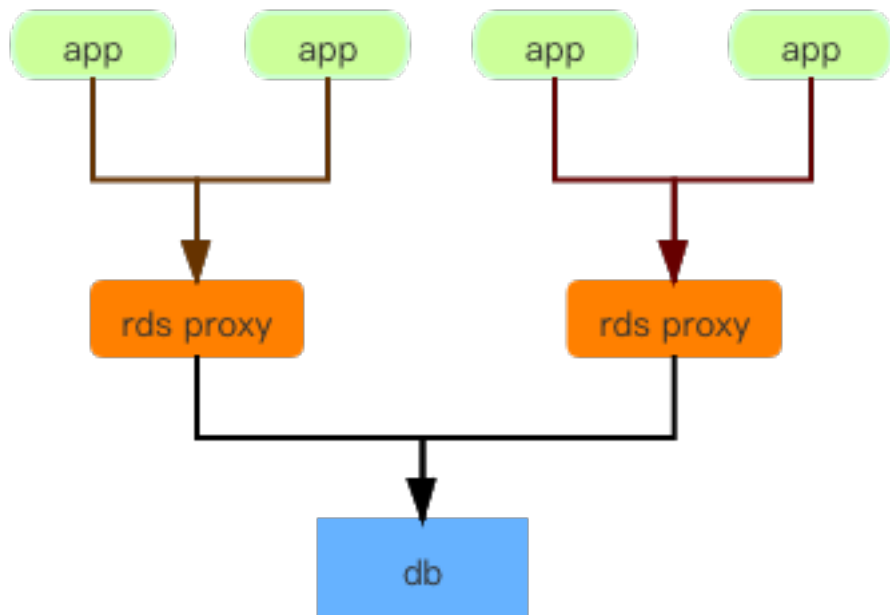


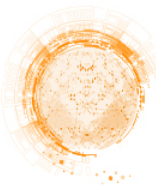
```
sysctl -w net.ipv4.tcp_delayed_ack=0
```



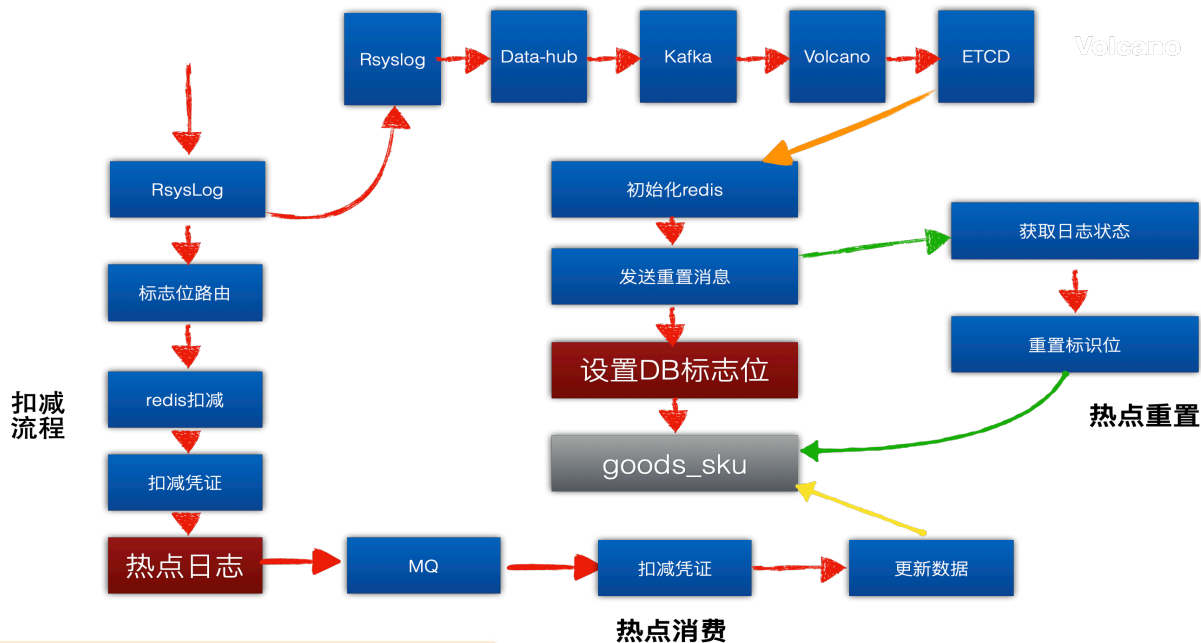
如何解决 - 应用层

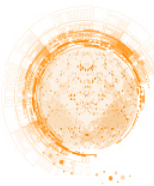
- 限流降级 热点探测
- 控制并发 排队
- 异步扣减 WAL (Redis + MySQL log)
- 合并更新 减少锁竞争



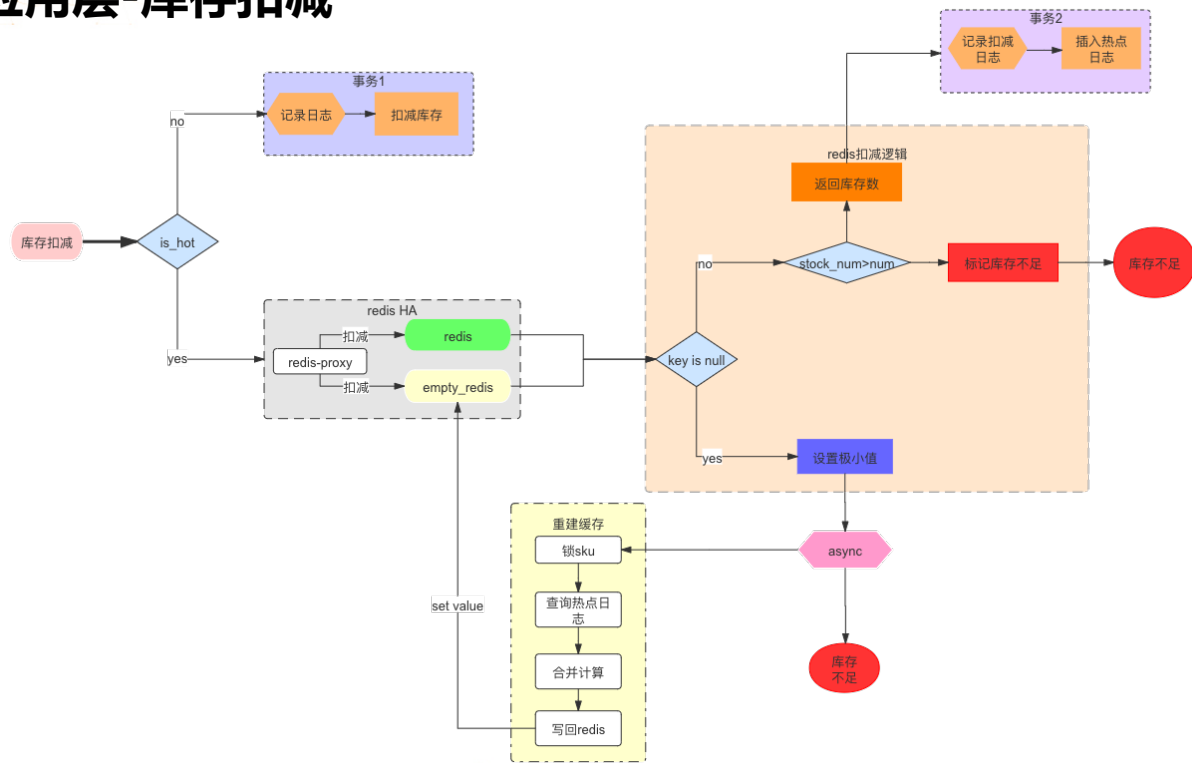


应用层-库存扣减



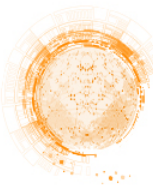


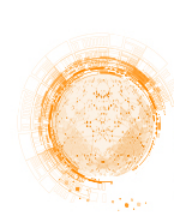
应用层-库存扣减



如何解决 - 应用层

- Redis 定位 防超卖，可能少卖
- 事务控制 非严格事务，二阶段补偿
- 并发处理 锁 + CAS





遗留问题

- 热点发现流程过长，交易转化率降低
- 热点日志合并按照sku维度处理，存在热点写
- 数据实时一致性弱



总结

- 全链路压测工具发现瓶颈
- 水平/垂直拆分
- 挖掘细节优化点

THANKS

